

UNIVERSIDAD DE LEÓN

Escuela de Ingenierías Industrial, Informática y Aeroespacial

**GRADO EN INGENIERÍA DE DATOS E
INTELIGENCIA ARTIFICIAL**

Trabajo de Fin de Grado

**Sistemas de Movilidad Autónoma en
Simuladores Urbanos mediante Redes
Neuronales y Algoritmos Evolutivos**

Autonomous Mobility Systems in Urban Simulators Using
Neural Networks and Evolutionary Algorithms

Autor: Sergio Alves Martín
Tutor/es: Sergio Rubio Martín / Alicia Merayo Corcoba

Junio, 2026

Ficha del trabajo

Alumno:	Sergio Alves Martín
Tutor/es:	Sergio Rubio Martín / Alicia Merayo Corcoba
Título:	Sistemas de Movilidad Autónoma en Simuladores Urbanos mediante Redes Neuronales y Algoritmos Evolutivos
Title:	Autonomous Mobility Systems in Urban Simulators Using Neural Networks and Evolutionary Algorithms
Convocatoria:	Junio, 2026

RESUMEN

Este Trabajo de Fin de Grado aborda el diseño, desarrollo, evolución y evaluación de sistemas de inteligencia artificial para movilidad autónoma en un simulador urbano realista construido sobre Unreal Engine 5. El proyecto se integra en la línea de trabajo de *CognitiveCarSimulatorReloaded*, una herramienta orientada a la rehabilitación y valoración cognitiva de pacientes con daño cerebral adquirido en colaboración con ASPAYM. La contribución principal consiste en dotar al entorno de tráfico autónomo capaz de circular por una ciudad virtual, interpretar sensores simulados, responder ante elementos normativos de tráfico y generar situaciones dinámicas que incrementen el realismo de la experiencia.

El desarrollo parte de una versión previa del simulador y evoluciona desde aproximaciones deterministas de navegación, como *Path Finding* y NavMesh, hacia una arquitectura basada en percepción local mediante raycasting, redes neuronales y algoritmos evolutivos. Cada vehículo autónomo procesa información del entorno mediante once sensores de distancia, variables cinemáticas y señales de contexto asociadas a semáforos, señales de stop y ceda el paso. La red neuronal empleada transforma estas entradas en dos salidas continuas, dirección y aceleración/frenado, que son aplicadas posteriormente por el sistema físico del vehículo. El entrenamiento se realiza mediante un gestor evolutivo que crea poblaciones de agentes, evalúa su comportamiento con una función de aptitud instrumentada, conserva los mejores pesos, aplica mutaciones y persiste los modelos más prometedores para sesiones posteriores.

Además del núcleo de IA, se han desarrollado sistemas de regulación del tráfico, validación de cumplimiento normativo, generación de métricas en CSV y análisis

estadístico mediante Python. Los resultados experimentales muestran una mejora sustancial frente a iteraciones anteriores: en la sesión de entrenamiento del 2 de junio de 2026 se alcanzó un mejor fitness de 117642.98 y se eliminó el porcentaje de fitness medio negativo; en la prueba final analizada el modelo alcanzó un mejor fitness de 84046.79, un tiempo medio de vida de 54.68 segundos y un 30 % de aciertos sobre 52 puntos de aparición. Aunque persisten limitaciones en intersecciones complejas y en ciertos puntos de aparición, el trabajo establece una base funcional y medible para tráfico autónomo en el simulador.

Palabras clave: simulación urbana, movilidad autónoma, redes neuronales, algoritmos evolutivos, Unreal Engine, raycasting, rehabilitación cognitiva.

ABSTRACT

This Bachelor's Thesis addresses the design, development, evolution and evaluation of artificial intelligence systems for autonomous mobility within a realistic urban driving simulator built on Unreal Engine 5. The project is part of the *CognitiveCarSimulatorReloaded* line of work, a tool aimed at cognitive rehabilitation and assessment of patients with acquired brain injury in collaboration with ASPAYM. The main contribution is to provide the simulator with autonomous traffic able to drive through a virtual city, interpret simulated sensors, react to traffic control elements and generate dynamic situations that increase the realism of the experience.

The work starts from a previous version of the simulator and evolves from deterministic navigation approaches, such as Path Finding and NavMesh, towards an architecture based on local perception through raycasting, neural networks and evolutionary algorithms. Each autonomous vehicle processes environmental information using eleven distance sensors, kinematic variables and contextual signals related to traffic lights, stop signs and yield signs. The neural network transforms these inputs into two continuous outputs, steering and throttle/braking, which are then applied by the physical vehicle system. Training is managed by an evolutionary controller that spawns populations of agents, evaluates their behavior using an instrumented fitness function, preserves the best weights, applies mutations and persists the most promising models for later sessions.

In addition to the AI core, the project includes traffic regulation systems, rule-compliance validation, CSV metric generation and statistical analysis through Python. The experimental results show a significant improvement over previous iterations: in the training session from June 2, 2026, the system reached a best fitness of 117642.98 and eliminated negative mean fitness; in the final analyzed test, the model achieved a best fitness of 84046.79, an average lifetime of 54.68 seconds and a 30 % success rate over 52 spawn points. Although limitations remain in complex intersections and

specific spawn points, the work establishes a functional and measurable foundation for autonomous traffic within the simulator.

Keywords: urban simulation, autonomous mobility, neural networks, evolutionary algorithms, Unreal Engine, raycasting, cognitive rehabilitation.

Índice general

Ficha del trabajo	I
Resumen	I
Abstract	II
Glosario	IX
1. Estudio del problema	1
1.1. Contexto	1
1.2. Problema que se resuelve	1
1.3. Objetivo general	2
1.4. Objetivos específicos	2
1.5. Estructura del trabajo	3
2. Estado del arte	4
2.1. Simuladores de conducción	4
2.2. Agentes autónomos en entornos urbanos	4
2.3. Redes neuronales y neuroevolución	5
2.4. Retos específicos	5
3. Tecnologías y herramientas	6
3.1. Unreal Engine 5	6
3.2. Sistema de físicas Chaos Vehicle	6
3.3. Raycasting y percepción local	6
3.4. Red neuronal	7
3.5. Algoritmos evolutivos	7
3.6. Análisis de datos	8
3.7. Control de versiones	8
4. Gestión del proyecto	9
4.1. Metodología	9
4.2. Cronología de decisiones	9
4.3. Alcance	10
4.4. Gestión de riesgos	11
4.5. Estimación de recursos y presupuesto	11

5. Núcleo del trabajo	13
5.1. Descripción general de la solución	13
5.2. Subsistemas	13
5.2.1. Gestor evolutivo	13
5.2.2. Controlador de IA y red neuronal	14
5.2.3. Percepción mediante sensores	14
5.2.4. Sistema de fitness	15
5.2.5. Semáforos	15
5.2.6. Stop y ceda el paso	16
5.3. Pipeline experimental	16
6. Resultados	17
6.1. Datos analizados	17
6.2. Resumen cuantitativo	17
6.3. Evolución del entrenamiento	18
6.4. Evaluación de test	18
6.5. Razones de muerte	19
6.6. Comportamiento ante señalización	20
6.7. Dificultad por puntos de aparición	21
6.8. Conclusión experimental	21
7. Normativa y accesibilidad	22
7.1. Protección de datos	22
7.2. Seguridad y uso responsable	22
7.3. Accesibilidad	22
8. Conclusiones y líneas futuras	24
8.1. Conclusiones	24
8.2. Aportaciones realizadas	24
8.3. Problemas encontrados	25
8.4. Líneas futuras	25
Agradecimientos	27
Bibliografía	28
A. Fuentes analizadas	29
A.1. Documentación administrativa y de formato	29
A.2. Actas de reunión	29
A.3. Descripciones del programa	29

A.4. Resultados experimentales	30
--	----

Índice de figuras

6.1.	Tendencia de fitness y tiempo medio en la sesión de entrenamiento del 2 de junio de 2026. Fuente: análisis generado a partir de CSV del proyecto.	18
6.2.	Resumen visual de la sesión de test final. Fuente: análisis generado a partir de CSV del proyecto.	19
6.3.	Distribución de razones de muerte en la sesión de test final. Fuente: análisis generado a partir de CSV del proyecto.	20
6.4.	Ranking de puntos de aparición por dificultad en test. Fuente: análisis generado a partir de CSV del proyecto.	21

Índice de tablas

4.1. Principales hitos y decisiones documentadas en reuniones.	9
4.2. Riesgos principales del proyecto.	11
6.1. Comparación de resultados principales en las sesiones finales analizadas.	17

Glosario

ASPAYM	Asociación/Fundación colaboradora orientada a mejorar la calidad de vida de personas con lesión medular, gran discapacidad física y daño cerebral adquirido.
Blueprint	Sistema visual de programación de Unreal Engine empleado para implementar lógica de juego, interacción, controladores y sistemas de simulación.
Chaos Vehicle	Sistema de físicas de Unreal Engine 5 utilizado para simular el comportamiento dinámico del vehículo.
DCA	Daño Cerebral Adquirido.
Fitness	Función de aptitud que cuantifica el comportamiento de cada vehículo autónomo durante una generación evolutiva.
NavMesh	Malla de navegación utilizada por motores de videojuegos para calcular rutas transitables. En este proyecto fue estudiada y posteriormente descartada como solución principal para vehículos.
NPC	<i>Non-Player Character</i> ; agente controlado por el sistema y no por el usuario.
Raycasting	Técnica de consulta geométrica que lanza rayos o trazas desde un actor para detectar obstáculos y distancias en el entorno.
RN	Red neuronal artificial.
UE5	Unreal Engine 5.

Capítulo 1

Estudio del problema

1.1 CONTEXTO

La conducción es una actividad compleja que exige coordinación motora, atención sostenida, atención dividida, interpretación de señales, percepción espacial y toma de decisiones bajo restricciones temporales. En personas que han sufrido daño cerebral adquirido, estas capacidades pueden verse alteradas aunque existan adaptaciones físicas que permitan manejar un vehículo. Por ello, antes de exponer al paciente a situaciones reales de tráfico, resulta de interés disponer de entornos controlados donde observar, entrenar y valorar comportamientos de conducción de forma segura.

El proyecto *CognitiveCarSimulatorReloaded* nace dentro de esta necesidad. La propuesta original define un simulador realista de conducción urbana para entrenamiento y valoración de alteraciones cognitivas tras un daño cerebral adquirido, desarrollado en colaboración con ASPAYM. La versión inicial del simulador, desarrollada durante el curso 2024-2025, ya incluía un entorno urbano, vehículo, interacción física, señalización y registro de datos. Sin embargo, las evaluaciones posteriores identificaron una carencia clave: el escenario necesitaba tráfico autónomo y dinámico para trasladar al usuario a una situación más realista.

El presente TFG aborda esa carencia desde el ámbito de la Ingeniería de Datos e Inteligencia Artificial. El foco no se sitúa en crear un simulador desde cero, sino en diseñar e integrar modelos de movilidad autónoma capaces de aprender comportamientos de conducción dentro de un entorno urbano ya existente. El trabajo combina redes neuronales, algoritmos evolutivos, sensores simulados, control físico en tiempo real y análisis de datos experimentales.

1.2 PROBLEMA QUE SE RESUELVE

El problema técnico principal consiste en conseguir que agentes autónomos, inicialmente vehículos, circulen por un mapa urbano de forma suficientemente estable, medible y compatible con las normas básicas de tráfico. A diferencia de un recorrido prefijado, un agente autónomo debe responder a múltiples condiciones locales: límites de la calzada, obstáculos, intersecciones, semáforos, señales de stop, señales de ceda el

paso y cambios en su propio estado dinámico.

Durante el desarrollo se evaluaron soluciones deterministas de navegación, como el uso de NavMesh o recorridos mediante *Path Finding*. Estas aproximaciones resultaron útiles como punto de partida, pero presentaron limitaciones para el objetivo clínico y experimental del simulador: rutas rígidas, giros poco naturales, dificultad para representar la toma de decisiones local y menor capacidad de adaptación a situaciones imprevistas. Como consecuencia, el proyecto evolucionó hacia una solución basada en percepción local mediante sensores y aprendizaje evolutivo.

1.3 OBJETIVO GENERAL

El objetivo general del trabajo es desarrollar e integrar un sistema de movilidad autónoma para un simulador urbano de conducción, basado en redes neuronales y algoritmos evolutivos, que permita entrenar, evaluar y analizar vehículos autónomos dentro de Unreal Engine 5.

1.4 OBJETIVOS ESPECÍFICOS

Los objetivos específicos son:

- Analizar el estado inicial del simulador y las restricciones técnicas heredadas de la versión previa.
- Diseñar una arquitectura de percepción para vehículos autónomos basada en sensores simulados mediante raycasting.
- Implementar una red neuronal ligera capaz de transformar entradas sensoriales y de contexto en acciones de conducción.
- Diseñar un proceso de entrenamiento evolutivo que evalúe poblaciones de vehículos, seleccione individuos destacados y aplique mutaciones sobre sus pesos.
- Integrar señales de tráfico, semáforos, stops y cedas en la lógica de percepción, parada y validación de la IA.
- Construir una función de fitness interpretable, con componentes positivos y penalizaciones, que permita medir progreso, errores y comportamientos no deseados.
- Generar registros experimentales en CSV y analizarlos mediante scripts reproducibles para evaluar entrenamiento y test.
- Identificar limitaciones, riesgos y líneas futuras para ampliar la movilidad autónoma a perfiles de conductor, peatones, bicicletas y otros agentes.

1.5 ESTRUCTURA DEL TRABAJO

El documento se organiza en ocho capítulos principales. Tras este estudio del problema, el capítulo 2 resume el estado del arte y los retos asociados a simuladores, agentes autónomos y aprendizaje evolutivo. El capítulo 3 describe las tecnologías y herramientas utilizadas. El capítulo 4 presenta la gestión del proyecto, la evolución temporal, los riesgos y una estimación preliminar de recursos. El capítulo 5 desarrolla el núcleo técnico de la solución. El capítulo 6 recoge los resultados experimentales. El capítulo 7 trata normativa, protección de datos y accesibilidad. Finalmente, el capítulo 8 expone conclusiones, aportaciones y líneas futuras.

Capítulo 2

Estado del arte

2.1 SIMULADORES DE CONDUCCIÓN

Los simuladores de conducción permiten reproducir situaciones de tráfico en un entorno controlado, repetible y seguro. En el ámbito formativo se utilizan para entrenar maniobras, respuesta ante eventos inesperados y familiarización con normas de circulación. En el ámbito sanitario, su valor reside en que permiten observar capacidades cognitivas y motoras sin exponer al usuario a riesgos reales. La versión previa de *CognitiveCarSimulator* ya se apoyaba en esta idea: un entorno urbano en Unreal Engine donde registrar tiempos de reacción, velocidades, colisiones y datos exportables para especialistas.

Para que un simulador de rehabilitación resulte convincente, el entorno no puede limitarse a una vía vacía. El usuario debe compartir la escena con otros agentes que creen carga cognitiva: vehículos que se detienen, peatones que cruzan, ciclistas, señales y elementos dinámicos. La propuesta de *CognitiveCarSimulatorReloaded* identifica precisamente esa necesidad al plantear IAs conductoras configurables, peatones, bicicletas y animales como ampliaciones progresivas del simulador.

2.2 AGENTES AUTÓNOMOS EN ENTORNOS URBANOS

La navegación autónoma en videojuegos y simulación suele abordarse mediante técnicas deterministas, como grafos de rutas, splines o mallas de navegación. Estas técnicas son robustas cuando el objetivo es desplazar agentes por zonas transitables con trayectorias controladas, pero pueden producir comportamientos rígidos si se aplican directamente a vehículos físicos. Un coche no se desplaza como un personaje humano: tiene inercia, radio de giro, aceleración, frenada, fricción y restricciones de control continuas.

En este proyecto se comprobó esa diferencia durante las primeras iteraciones. El uso de NavMesh y Path Finding fue estudiado para permitir recorridos por la ciudad, pero las reuniones de seguimiento y los informes técnicos documentan problemas de rigidez, dificultad en intersecciones y falta de naturalidad. Por ello se adoptó una estrategia más cercana a la percepción local: el agente no sigue una ruta global cerrada,

sino que interpreta distancias, estado del tráfico y contexto normativo para producir acciones continuas.

2.3 REDES NEURONALES Y NEUROEVOLUCIÓN

Las redes neuronales artificiales permiten aproximar funciones complejas a partir de entradas numéricas. En este TFG se emplean como política de control: reciben sensores, velocidad, contexto de señalización y estado de parada, y producen dirección y aceleración/frenado. La arquitectura elegida es deliberadamente ligera para poder ejecutarse en tiempo real sobre múltiples vehículos: 22 entradas, una capa oculta de 16 neuronas y dos salidas continuas con activación hiperbólica.

El entrenamiento no se plantea como aprendizaje supervisado, ya que no se dispone de un conjunto amplio de trayectorias humanas etiquetadas dentro del simulador. En su lugar, se adopta un algoritmo evolutivo. Cada individuo de una población representa un conjunto de pesos de la red neuronal. La simulación evalúa el comportamiento mediante fitness; los mejores pesos se conservan, se mutan y se reutilizan en generaciones posteriores. Este enfoque resulta adecuado para explorar comportamientos en entornos donde la señal de aprendizaje procede de la interacción con el mundo simulado.

2.4 RETOS ESPECÍFICOS

Los principales retos identificados son:

- **Simulación física:** el control neuronal debe traducirse a entradas de un vehículo con físicas reales, evitando aceleraciones, frenadas o giros no plausibles.
- **Intersecciones:** los cruces concentran la mayor complejidad, porque combinan decisión de trayectoria, detección de límites, prioridades y otros vehículos.
- **Diseño de fitness:** una función de aptitud mal calibrada puede premiar estrategias no deseadas, como detenerse indefinidamente, avanzar demasiado despacio o compensar errores graves con comportamientos parciales.
- **Generalización por puntos de aparición:** el modelo debe funcionar en diferentes zonas del mapa, no únicamente en los puntos de entrenamiento más sencillos.
- **Coste computacional:** entrenar decenas de vehículos simultáneamente en Unreal Engine implica carga de CPU, GPU y física.
- **Trazabilidad:** al tratarse de un sistema experimental, es imprescindible registrar métricas suficientes para explicar por qué una iteración mejora o empeora.

Capítulo 3

Tecnologías y herramientas

3.1 UNREAL ENGINE 5

Unreal Engine 5 es el motor sobre el que se desarrolla el simulador. Proporciona renderizado 3D, editor visual, sistema de actores, colisiones, físicas, Blueprints y herramientas de depuración. Su elección viene heredada de la versión inicial del proyecto y resulta coherente con los requisitos de realismo visual, interacción en tiempo real y despliegue del simulador como aplicación ejecutable.

El trabajo se implementa principalmente mediante Blueprints, lo que permite integrar lógica de IA, controladores, sensores y eventos de tráfico dentro del propio editor. Este enfoque facilita la iteración rápida, aunque introduce retos de mantenibilidad cuando los grafos crecen. Por ello se han documentado los Blueprints principales y se han generado descripciones fechadas de módulos como BP_EvolutionManager, BP_AiController, BP_VehicleAI2, BP_TrafficLight, BP_Sign_Stop y BP_Sign_Yield.

3.2 SISTEMA DE FÍSICAS CHAOS VEHICLE

El vehículo se controla mediante el sistema de físicas de Unreal Engine. La red neuronal no mueve directamente el actor, sino que genera consignas continuas que se aplican al componente de movimiento del vehículo. La salida de dirección se envía como *steering input*; la salida de aceleración/frenado se interpreta de forma separada: valores positivos se aplican como acelerador y valores negativos como freno.

Esta separación es importante porque mantiene el comportamiento dentro de la física del motor. El agente aprende a actuar sobre el vehículo, pero la respuesta final depende de velocidad, fricción, inercia, contacto con el suelo y restricciones propias del modelo físico.

3.3 RAYCASTING Y PERCEPCIÓN LOCAL

La percepción del vehículo se implementa mediante trazas o rayos simulados. El controlador lanza once sensores en un ángulo de visión de 180 grados desde el vehículo.

Cada sensor devuelve una distancia normalizada al primer obstáculo relevante, con valor cercano a 1 cuando no se detecta un obstáculo próximo y valores menores cuando existe un límite, pared, vehículo o borde de intersección.

Además de los sensores laterales y frontales, se calculan variables derivadas como la distancia frontal a obstáculo, la distancia del sensor izquierdo, la distancia del sensor derecho y la diferencia horizontal entre ambos. Estas magnitudes permiten construir una señal de centrado en carril y detectar correcciones equivocadas.

3.4 RED NEURONAL

La red neuronal empleada es un perceptrón multicapa ligero. Su vector de entrada se compone de:

- velocidad longitudinal normalizada;
- velocidad angular normalizada;
- once lecturas de sensores;
- estado de semáforo activo;
- estado numérico del semáforo;
- distancia normalizada a la línea de parada;
- indicador de parada forzada;
- codificación *one-hot* del tipo de control de tráfico: semáforo, stop, ceda o ninguno;
- sesgo constante.

Esto produce 22 entradas. La capa oculta contiene 16 neuronas y utiliza tanh como función de activación. La capa de salida contiene dos neuronas, también con tanh, de modo que sus valores se encuentran en el rango $[-1, 1]$. La primera salida controla dirección y la segunda controla aceleración o frenado.

3.5 ALGORITMOS EVOLUTIVOS

El entrenamiento se basa en población, selección y mutación. El gestor evolutivo crea una población de vehículos, les asigna pesos iniciales o mutados, ejecuta la simulación y registra el fitness final de cada individuo. Los mejores pesos se guardan en disco como modelos reutilizables, distinguiendo entre el mejor modelo histórico y el mejor de sesión.

La tasa de mutación final documentada se sitúa en torno al 4 % para mutaciones ordinarias, combinando variaciones pequeñas y mutaciones grandes excepcionales. Esta estrategia busca mantener diversidad sin destruir completamente comportamientos ya útiles.

3.6 ANÁLISIS DE DATOS

El proyecto genera tres ficheros principales:

- **Fitness_Debug.csv**, con métricas por coche: fitness, tiempo de vida, punto de aparición, razón de muerte, penalizaciones, bonus, datos de conducción, validación de stops y cedas, mutaciones y familia genealógica.
- **Training_Summary.csv**, con resumen por generación de entrenamiento.
- **Test_Summary.csv**, con resumen por generación de test, incluyendo aciertos, errores y tasas.

El análisis posterior se realiza mediante Python, NumPy, Matplotlib y Seaborn. El script `analyse_current.py` valida la calidad de los CSV, detecta automáticamente si la sesión es de entrenamiento o test, genera informes textuales y produce gráficas de evolución, muertes, distribución de fitness, ranking por spawn, control de entradas, convergencia, diversidad, correlaciones y diagnósticos específicos de paradas o cedas. El script `clean_unreal_log.py` permite limpiar logs de Unreal Engine conservando errores, avisos relevantes, mensajes de Blueprint y trazas propias del proyecto.

3.7 CONTROL DE VERSIONES

El trabajo se ha desarrollado con Git/GitLab en un contexto colaborativo. La documentación de seguimiento recoge tareas de limpieza de repositorio, configuración de `.gitignore`, eliminación de binarios pesados y organización de issues y milestones. Esta disciplina ha sido relevante porque el proyecto combina assets de Unreal, Blueprints, datos generados y modelos guardados.

Capítulo 4

Gestión del proyecto

4.1 METODOLOGÍA

El desarrollo se ha organizado mediante un enfoque iterativo e incremental. Las reuniones de seguimiento se celebraron de forma periódica, generalmente cada dos jueves, con participación de tutores, equipo técnico y personas vinculadas al proyecto original. En estas reuniones se revisaban avances, problemas, decisiones de arquitectura y siguientes objetivos. Esta dinámica se aproxima a una metodología ágil por sprints, aunque adaptada al contexto académico y a la disponibilidad del equipo.

La planificación inicial contemplaba una progresión amplia: semáforos y señales, IA vehicular, stops y cedas, perfiles de conductor, peatones, bicicletas, animales, diversidad de assets y optimización. El desarrollo real concentró el esfuerzo en construir una base sólida de IA vehicular, debido a la dificultad técnica de lograr navegación autónoma estable en la ciudad.

4.2 CRONOLOGÍA DE DECISIONES

Tabla 4.1: Principales hitos y decisiones documentadas en reuniones.

Fecha	Hito o decisión
27/11/2025	Se revisa la integración de semáforos dinámicos y se fija como objetivo la IA vehicular. Se descarta una malla dinámica para reconocer caminos por los problemas previsibles de desarrollo.
11/12/2025	Se descarta el mapeo completo con splines como opción principal y se considera Path Finding de Unreal. Se decide que el modelo analice entorno y contexto, y que Unreal aplique físicamente la acción.
17/12/2025	Se revisan perfiles de conductor y se fijan provisionalmente dos salidas del modelo: rotación y velocidad, ambas en rango $[-1, 1]$.

08/01/2026	Se descarta Path Finding para vehículos y se valida la idea de entrenamiento evolutivo por prueba-error. El objetivo prioritario pasa a ser aprender básicos de conducción.
05/02/2026	Se discuten problemas en curvas e intersecciones. Se acepta temporalmente el etiquetado dinámico de bordes y se plantea la posibilidad de coordinar más de un modelo.
19/02/2026	Se fija como prioridad una demo técnica en dos semanas que demuestre red neuronal, sensores y bordes funcionando por todo el mapa.
05/03/2026	La demostración se clasifica como fallida y no concluyente. Se acuerda analizar exhaustivamente la función de fitness y sesgar escenarios complejos para facilitar aprendizaje.
19/03/2026	La nueva demostración se considera aceptable: el modelo recorre casi la totalidad de la mayoría de la ciudad y cerca de un 50 % sobrevive más de 100 segundos. Se planifica reorganización en GitLab, build Windows y siguientes mejoras.

4.3 ALCANCE

El alcance funcional completado se centra en IA vehicular autónoma. Incluye:

- controlador de IA capaz de aplicar dirección, aceleración y frenado sobre el vehículo físico;
- sensores por raycasting y entradas normalizadas para la red neuronal;
- entrenamiento evolutivo con población, mutación, elitismo y persistencia de modelos;
- integración con semáforos, señales de stop y ceda el paso;
- cálculo de fitness con componentes de avance, centrado, velocidad, cumplimiento normativo, penalizaciones y bonus;
- exportación de métricas y análisis automatizado de resultados;
- separación de puntos de aparición válidos para entrenamiento y test.

Quedan fuera del alcance final de esta primera memoria la implementación completa de peatones, bicicletas, animales y perfiles de conductor totalmente diferenciados. Estos elementos se mantienen como líneas futuras porque dependen de una base vehicular estable.

4.4 GESTIÓN DE RIESGOS

Tabla 4.2: Riesgos principales del proyecto.

Riesgo	Impacto	Probabilidad	Mitigación
Rigidez de navegación determinista	Comportamiento poco realista y baja utilidad clínica	Alta	Pivotar a percepción local con sensores y red neuronal.
Fitness mal calibrado	Aprendizaje de estrategias no deseadas	Alta	Instrumentar componentes, exportar CSV y analizar penalizaciones por familia de muerte.
Coste computacional	Entrenamientos lentos y dependencia de GPU/CPU	Media	Poblaciones moderadas, análisis offline y propuesta futura de modo sin renderizado.
Complejidad de intersecciones	Colisiones y violaciones normativas en cruces	Alta	Bordes dinámicos, zonas de cruce, validación de stops/cedas y separación train/test.
Sobreajuste a spawns concretos	Buen rendimiento local pero mala generalización	Media	Normalización por punto de aparición y test con spawns diferenciados.
Pérdida de conocimiento entrenado	Reinicio de aprendizaje entre sesiones	Media	Persistencia de modelos <code>SuperCarModel</code> y <code>SessionCarModel</code> .

4.5 ESTIMACIÓN DE RECURSOS Y PRESUPUESTO

El trabajo se desarrolló en el marco de prácticas y TFG, por lo que no existe una contratación externa directa asociada. No obstante, desde el punto de vista de proyecto técnico, los recursos empleados son:

- equipo de desarrollo con capacidad para Unreal Engine 5;
- licencia y entorno de Unreal Engine;

- repositorio Git/GitLab;
- herramientas Python de análisis;
- tiempo de desarrollo, documentación, reuniones y experimentación.

La estimación económica debe cerrarse con los criterios del centro y del tutor antes de la entrega final. Como base para completar el presupuesto, se propone separar coste humano, coste hardware amortizado, coste software y coste de infraestructura. En este proyecto, el coste software directo es reducido al apoyarse en herramientas sin coste de licencia para el desarrollo académico, mientras que el coste principal corresponde a horas de ingeniería y experimentación.

Capítulo 5

Núcleo del trabajo

5.1 DESCRIPCIÓN GENERAL DE LA SOLUCIÓN

La solución final se estructura como un sistema de agentes autónomos entrenados dentro del simulador. Cada vehículo dispone de un controlador de IA que percibe el entorno, ejecuta una red neuronal, aplica acciones al sistema físico y registra métricas de comportamiento. Por encima de los vehículos existe un gestor evolutivo que crea poblaciones, asigna pesos, evalúa resultados, selecciona los mejores individuos y guarda modelos.

El sistema se completa con elementos de tráfico que comunican contexto a los controladores: semáforos, señales de stop, señales de ceda y zonas de cruce. Estos elementos no son meros objetos visuales, sino actores con lógica propia que activan zonas de detección, informan al vehículo de líneas de parada, controlan estados y aplican penalizaciones o bonus según el comportamiento observado.

5.2 SUBSISTEMAS

5.2.1 Gestor evolutivo

`BP_EvolutionManager` coordina entrenamiento y test. Sus responsabilidades principales son:

- cargar modelos guardados desde `SuperCarModel` o inicializar pesos si no existen;
- refrescar los puntos de aparición válidos según el modo de entrenamiento o test;
- generar poblaciones de vehículos;
- asignar cerebros aleatorios, cargados o mutados;
- detectar fin de generación cuando todos los vehículos han muerto o finalizado;
- ordenar por fitness, normalizar por spawn y seleccionar el mejor individuo;
- guardar pesos de sesión e histórico;

- exportar resúmenes de entrenamiento y test.

Una decisión relevante es la normalización por punto de aparición. La ciudad contiene zonas de dificultad desigual; comparar directamente fitness de spawns distintos puede favorecer rutas sencillas. Por ello se mantiene el mejor fitness observado por spawn y se selecciona el mejor coche para pesos atendiendo a una relación normalizada respecto a ese punto.

5.2.2 Controlador de IA y red neuronal

`BP_AiController` actúa como intermediario entre red neuronal y vehículo físico. En cada tick comprueba que el vehículo controlado existe y no está muerto, prepara las entradas de red, ejecuta la inferencia y aplica las salidas al componente de movimiento.

Para evitar saltos bruscos, el controlador dispone de variables de suavizado como `MaxSteeringRate` y `MaxThrottleRate`. La versión documentada mantiene un seguimiento estable entre objetivo y entrada aplicada, con gap medio cero en los informes finales, lo que indica que la señal de control se está transmitiendo sin divergencias instrumentales.

La inferencia se calcula manualmente sobre arrays de pesos:

$$h_j = \tanh \left(\sum_i w_{j,i}^{IH} x_i \right)$$

$$y_k = \tanh \left(\sum_j w_{k,j}^{HO} h_j \right)$$

donde x_i son las entradas normalizadas, h_j las neuronas ocultas e y_k las dos salidas de control. El sistema incluye guardas para detectar cerebros inválidos: número incorrecto de entradas, pesos ausentes o dimensiones incoherentes. Si el error se repite, el coche se elimina con razón de muerte `INVALID_BRAIN`.

5.2.3 Percepción mediante sensores

El controlador lanza once trazas en abanico sobre 180 grados. Cada sensor devuelve una distancia normalizada hasta el primer elemento relevante. Los bordes de intersección se tratan con lógica específica: solo deben considerarse obstáculos cuando no corresponden a la decisión de dirección o trigger actual. Esta solución evita que el vehículo interprete como barrera una zona por la que realmente debe circular al completar una maniobra.

Las entradas sensoriales se combinan con información de tráfico. Si el vehículo entra en la zona de un semáforo, stop o ceda, el actor correspondiente comunica

al controlador el tipo de control, la línea de parada y la distancia de entrada. Esta mezcla de percepción geométrica y contexto normativo permite que la red no dependa únicamente de distancia a obstáculos.

5.2.4 Sistema de fitness

La función de fitness se ha convertido en uno de los elementos centrales del proyecto. Su objetivo no es solo premiar distancia o velocidad, sino guiar comportamientos compatibles con conducción urbana. La versión final documentada incluye:

- recompensa por avance, velocidad útil y centrado respecto a sensores laterales;
- recompensa por mantener una relación coherente entre velocidad y distancia frontal;
- penalización por exceso de velocidad ante obstáculos;
- penalización por corregir en sentido contrario cuando el vehículo está descentrado;
- penalización por marcha atrás, estancamiento, salida de calzada y colisiones;
- penalización por violar semáforos, stops, cedas y zonas no navegables;
- bonus por completar correctamente stops y cedas;
- métricas acumuladas de frenado, aceleración, espera, steering y contexto de parada.

La instrumentación permite explicar por qué muere cada coche y qué componentes dominan su fitness. Esto fue decisivo tras la demostración fallida del 5 de marzo de 2026, cuando se acordó analizar el fitness con detalle para detectar cálculos o restricciones incorrectas.

5.2.5 Semáforos

El sistema de semáforos se compone de `BP_IntersectionController` y `BP_TrafficLight`. El controlador alterna fases norte-sur y este-oeste, incluyendo verde, ámbar y todo rojo. Cada semáforo mantiene materiales dinámicos para representar estado visual y una zona de sensor que detecta vehículos.

Cuando un vehículo entra en la zona, el semáforo comunica estado, línea de parada y distancia. Si el semáforo está en rojo o ámbar restrictivo, el controlador activa contexto de parada. Al salir de la zona, se comprueba si el vehículo atravesó la línea a una velocidad legal. El sistema aprende umbrales de velocidad legal a partir de muestras previas, evitando depender siempre de un valor fijo.

5.2.6 Stop y ceda el paso

Las señales `BP_Sign_Stop` y `BP_Sign_Yield` amplían la interacción normativa. En stop se exige detención suficiente dentro de una ventana dinámica antes de la línea, espera mínima y comprobación de cruce libre. Si se cumple, se aplica bonus y se libera el vehículo; si no, se aplica penalización y muerte por `STOP_VIOLATION`. En ceda se monitoriza velocidad, aproximación y disponibilidad de la zona de cruce, con penalización por `YIELD_VIOLATION` y bonus al completar correctamente.

La diferencia entre stop y ceda es metodológicamente importante. El stop requiere detención explícita; el ceda permite continuar si la velocidad y la prioridad son compatibles con la seguridad del cruce.

5.3 PIPELINE EXPERIMENTAL

Cada ejecución genera registros en CSV. El análisis posterior produce informes y gráficas. Este pipeline permite comparar sesiones, detectar regresiones y localizar spawns problemáticos. En las últimas sesiones se emplearon métricas como:

- *BestFit* máximo y medio;
- *MeanFit* por generación;
- tiempo medio de vida;
- porcentaje de fitness medio negativo;
- razones de muerte por familia;
- fallos legales;
- bonus de stop y ceda;
- distribución por puntos de aparición;
- correlaciones entre componentes y fitness final;
- estabilidad de steering y throttle.

El resultado es un ciclo de desarrollo cerrado: implementar, entrenar, exportar, analizar, diagnosticar y volver a ajustar.

Capítulo 6

Resultados

6.1 DATOS ANALIZADOS

La evaluación más reciente disponible se compone de una sesión de entrenamiento `train_2026-06-02_12-00` y una sesión de test `test_2026-06-02_13-17`. Ambas fueron procesadas por el script de análisis, que validó la coherencia entre `Summary` y `Debug`, sin alertas fuertes de desalineación.

La sesión de entrenamiento contiene 607 generaciones y 19424 registros de vehículos. La sesión de test contiene 5 generaciones y 260 registros, correspondientes a 52 puntos de aparición evaluados por generación.

6.2 RESUMEN CUANTITATIVO

Tabla 6.1: Comparación de resultados principales en las sesiones finales analizadas.

Métrica	Entrenamiento	Test
Generaciones	607	5
Registros debug	19424	260
BestFit máximo	117642.98	84046.79
MeanFit medio	17404.52	33600.03
MeanFit mediano	16445.88	34181.57
Tiempo medio de vida	41.01 s	54.68 s
MeanFit negativo	0.0 %	0.0 %
Razón de muerte principal	TIMEFINISHED (21 %)	TIMEFINISHED (30 %)
Fallo legal	6.82 %	1.92 %
LAZY	0.00 %	0.00 %
Tasa de acierto	—	30.00 %

El entrenamiento final mejora de forma clara respecto a sesiones anteriores en estabilidad de fitness medio. El análisis comparativo lo sitúa como la mejor sesión entre las nueve últimas en *MeanFit* medio y en menor porcentaje de fitness negativo.

La sesión previa del 1 de junio había alcanzado mayor *BestFit* máximo, pero con peor fitness medio y más dispersión; por tanto, la sesión del 2 de junio resulta más equilibrada.

6.3 EVOLUCIÓN DEL ENTRENAMIENTO

En entrenamiento, el mejor fitness se alcanza en la generación 205 con 117642.98. El fitness medio máximo se alcanza en la generación 173 con 34896.93. Aunque el informe detecta tendencia media negativa si se analiza la serie completa, también observa tendencia positiva reciente y ausencia de convergencia clara, lo que indica que el sistema todavía explora soluciones.

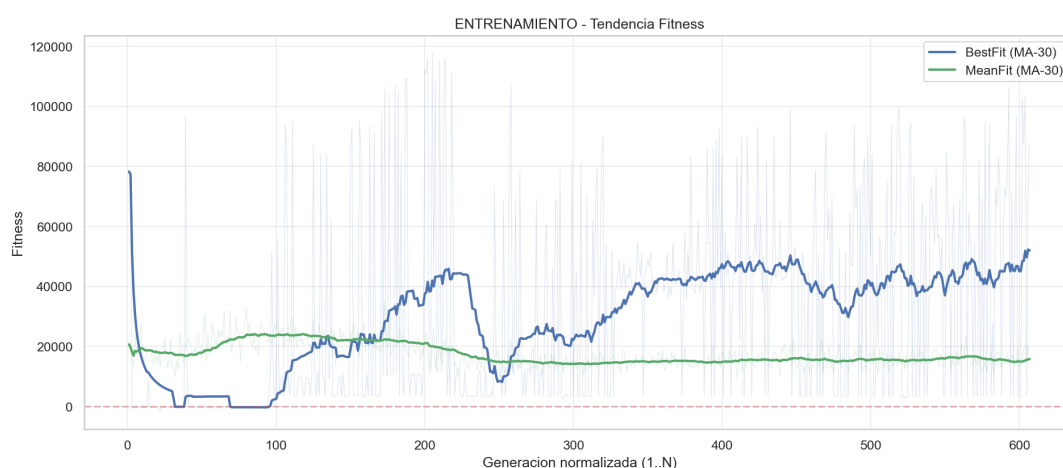


Figura 6.1: Tendencia de fitness y tiempo medio en la sesión de entrenamiento del 2 de junio de 2026. Fuente: análisis generado a partir de CSV del proyecto.

Un resultado importante es la desaparición de muertes por estancamiento **LAZY**. En sesiones anteriores esta causa llegó a dominar parte del comportamiento, indicando agentes que aprendían a no moverse o a avanzar de forma insuficiente. En la sesión final, **LAZY** representa 0 de 19424 registros.

6.4 EVALUACIÓN DE TEST

El test final muestra un mejor fitness de 84046.79 en la generación 5 y un fitness medio de 33600.03. La tasa de acierto promedio es del 30 %, con intervalo de confianza global aproximado del 24.75 % al 35.83 %. El valor debe interpretarse con cautela porque la muestra contiene cinco generaciones, pero la comparación con sesiones anteriores es positiva: el test final ocupa el primer puesto entre las nueve últimas sesiones tanto en *BestFit* como en *MeanFit*, y mejora 24.23 puntos porcentuales de acierto respecto al test inmediatamente anterior.

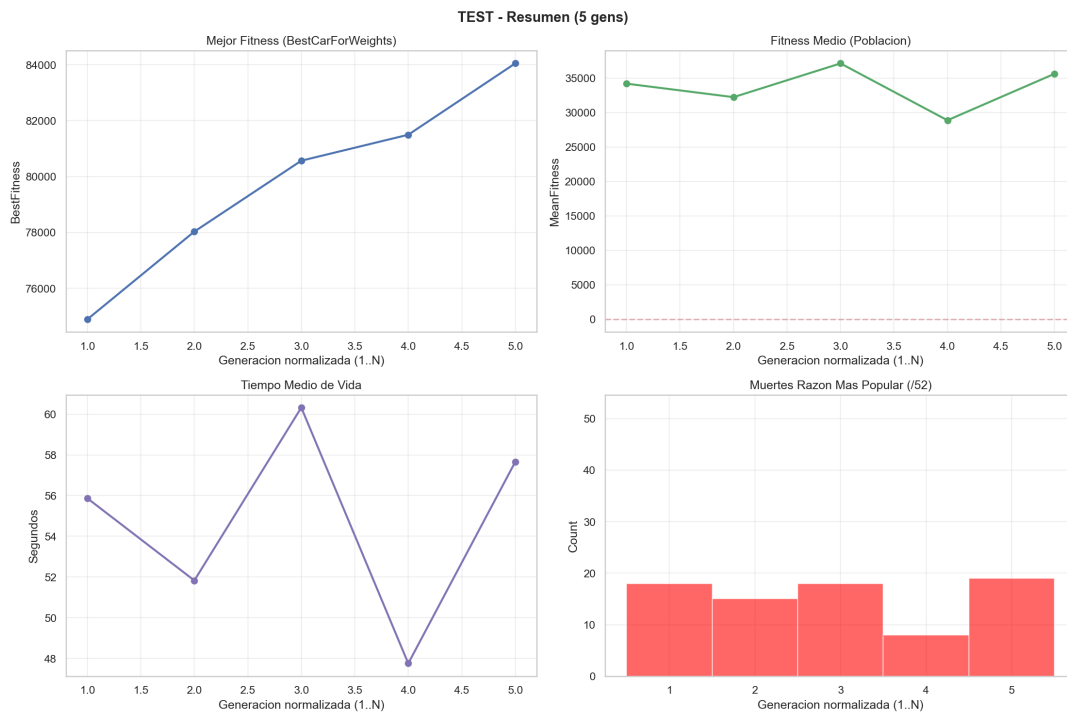


Figura 6.2: Resumen visual de la sesión de test final. Fuente: análisis generado a partir de CSV del proyecto.

6.5 RAZONES DE MUERTE

En test, las razones de muerte por familia son:

- colisiones: 177 registros, 68.08 %;
- finalización por tiempo: 78 registros, 30.00 %;
- violaciones normativas: 5 registros, 1.92 %.

Este resultado muestra que el principal problema restante ya no es la falta de movimiento ni la infracción sistemática de señales, sino la robustez geométrica en determinadas zonas del mapa. Las colisiones se concentran en bordes de intersección y volúmenes de navegación concretos, lo que apunta a mejorar etiquetado de bordes, sensores locales y escenarios de entrenamiento por spawn.

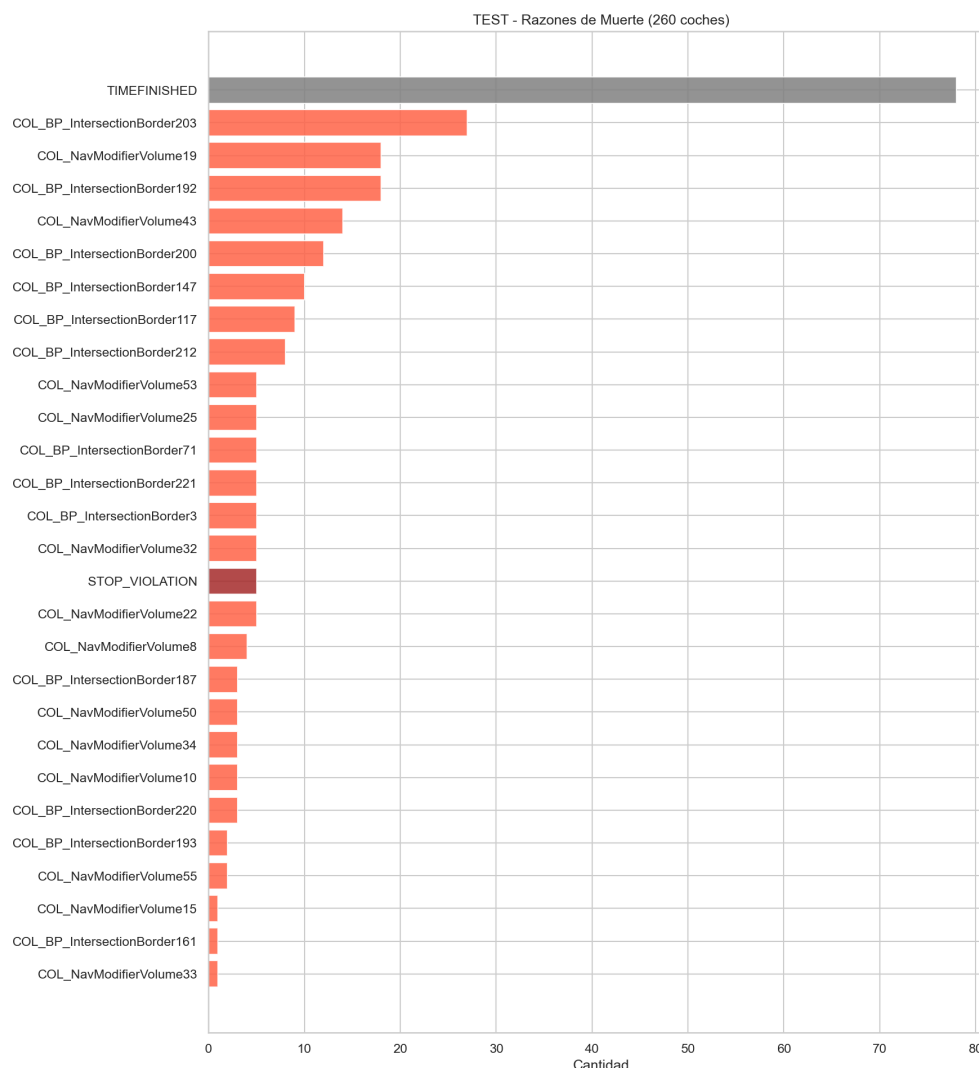


Figura 6.3: Distribución de razones de muerte en la sesión de test final. Fuente: análisis generado a partir de CSV del proyecto.

6.6 COMPORTAMIENTO ANTE SEÑALIZACIÓN

Los resultados muestran que, cuando el vehículo entra en contexto de stop o semáforo, frena de forma proporcionalmente superior a la conducción global. En test, la proporción de frenado en contexto de parada alcanza el 99.56%, frente al 21.42% global. Además, los fallos legales se reducen al 1.92% de los registros, señal de que la integración de stop, ceda y semáforo funciona como mecanismo de control.

Persisten, no obstante, candidatos a atasco en stop y ceda. En el test final se detectan 3 candidatos fuertes de atasco en stop sobre 78 TIMEFINISHED y 1 candidato de atasco en ceda. Estos casos no invalidan la solución, pero señalan zonas para depuración posterior.

6.7 DIFICULTAD POR PUNTOS DE APARICIÓN

El análisis por spawn revela diferencias importantes. En test, BP_SpawnPoint51 obtiene la mayor media de fitness, con 75303.3, mientras BP_SpawnPoint24 presenta la peor media, con -62.1 y 80% de valores negativos. Otros puntos difíciles son BP_SpawnPoint10, BP_SpawnPoint5, BP_SpawnPoint39 y BP_SpawnPoint23.

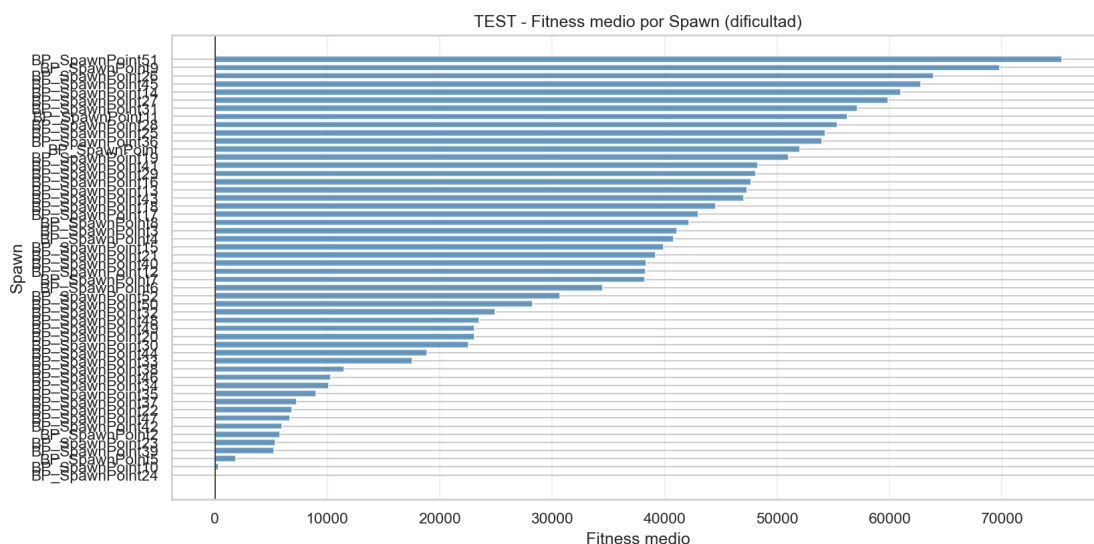


Figura 6.4: Ranking de puntos de aparición por dificultad en test. Fuente: análisis generado a partir de CSV del proyecto.

Esta dispersión confirma la necesidad de mantener evaluación por spawn y no limitarse a métricas globales. Un modelo puede mostrar buen rendimiento medio y, aun así, fallar de forma sistemática en zonas concretas de la ciudad.

6.8 CONCLUSIÓN EXPERIMENTAL

La evaluación confirma que la arquitectura propuesta es funcional y medible. El sistema aprende comportamientos útiles, evita el estancamiento, responde a contextos de parada y alcanza supervivencias significativas en múltiples puntos del mapa. Sin embargo, todavía no puede considerarse una IA de tráfico completamente finalizada: las colisiones en intersecciones y la variabilidad por spawn requieren más entrenamiento dirigido, mejoras de percepción y validación con escenarios más amplios.

Capítulo 7

Normativa y accesibilidad

7.1 PROTECCIÓN DE DATOS

El proyecto se orienta a una aplicación potencial en rehabilitación cognitiva, por lo que debe contemplar desde el diseño la protección de datos personales y, especialmente, de datos de salud. Durante el desarrollo descrito en esta memoria no se ha trabajado con datos reales de pacientes. Los registros analizados corresponden a vehículos autónomos simulados, métricas de entrenamiento y resultados técnicos.

En una futura validación clínica, cualquier uso con personas usuarias deberá ajustarse al Reglamento General de Protección de Datos y a la normativa española aplicable. Será necesario definir responsable del tratamiento, finalidad, base jurídica, minimización de datos, conservación, consentimiento informado cuando proceda, medidas de seudonimización y control de acceso. También deberá diferenciarse entre datos necesarios para la evaluación terapéutica y métricas técnicas internas del simulador.

7.2 SEGURIDAD Y USO RESPONSABLE

El simulador no sustituye por sí mismo una evaluación clínica ni una autorización administrativa para conducir. Su función debe entenderse como apoyo a profesionales, entrenamiento controlado y herramienta de observación. Cualquier conclusión sobre capacidad real de conducción debería depender de especialistas y procedimientos establecidos.

Desde el punto de vista técnico, el sistema autónomo genera tráfico artificial. Por tanto, sus fallos no suponen riesgo físico directo durante el entrenamiento virtual, pero sí pueden afectar a la validez de la experiencia. Es importante que las sesiones con pacientes utilicen modelos suficientemente probados para evitar comportamientos incoherentes que confundan al usuario o introduzcan estímulos no deseados.

7.3 ACCESIBILIDAD

El proyecto se dirige a personas con posibles alteraciones cognitivas y físicas. Por ello, la accesibilidad debe contemplarse en varias capas:

- interfaces claras, con instrucciones sencillas y ausencia de sobrecarga visual;
- compatibilidad con periféricos adaptados, como volante, pedales o mandos específicos;
- configuración de dificultad, densidad de tráfico y tipos de eventos;
- posibilidad de repetir escenarios concretos para entrenamiento progresivo;
- exportación de resultados comprensible para profesionales;
- control de mareo o fatiga mediante rendimiento estable y duración ajustable.

El trabajo actual contribuye especialmente a la configuración de dificultad mediante tráfico autónomo y eventos normativos. A futuro, la accesibilidad debe integrarse con perfiles de paciente, parámetros de escenario y supervisión profesional.

Capítulo 8

Conclusiones y líneas futuras

8.1 CONCLUSIONES

El trabajo realizado demuestra la viabilidad de integrar movilidad autónoma basada en redes neuronales y algoritmos evolutivos dentro de un simulador urbano en Unreal Engine 5. La solución final supera las limitaciones iniciales de navegación determinista y proporciona una arquitectura capaz de percibir el entorno, actuar sobre un vehículo físico, aprender mediante generaciones y producir datos analizables.

La principal aportación técnica es la combinación de cuatro piezas: percepción local por raycasting, red neuronal ligera, gestor evolutivo con persistencia de modelos y función de fitness instrumentada. Esta combinación permite entrenar agentes sin disponer de un dataset previo de conducción humana y, al mismo tiempo, analizar con detalle por qué cada iteración mejora o falla.

También se ha demostrado la importancia de la trazabilidad. Las decisiones más relevantes del proyecto no surgieron de una única implementación, sino de iteraciones documentadas: descarte de mallas dinámicas, prueba y abandono de Path Finding como solución principal, introducción de bordes dinámicos, reformulación del fitness tras una demo fallida y separación entre entrenamiento y test.

Los resultados finales son prometedores pero no definitivos. La sesión final de entrenamiento alcanza buen fitness máximo, elimina fitness medio negativo y erradica el estancamiento tipo LAZY. La sesión final de test mejora claramente a las anteriores y alcanza un 30 % de acierto. Aun así, las colisiones siguen siendo la familia de muerte dominante, por lo que la IA todavía requiere mejoras antes de considerarse tráfico autónomo plenamente robusto para sesiones clínicas.

8.2 APORTACIONES REALIZADAS

Las aportaciones principales son:

- diseño de arquitectura de IA vehicular con 22 entradas y dos salidas continuas;
- sensores por raycasting adaptados a límites, obstáculos e intersecciones;
- entrenamiento evolutivo con población, mutación, elitismo, pesos cargados y pesos

de sesión;

- normalización de fitness por punto de aparición;
- integración de semáforos, stops y cedas con contexto de parada y validación;
- función de fitness con componentes de avance, centrado, velocidad, señales, penalizaciones y bonus;
- persistencia de modelos entrenados;
- pipeline de análisis con informes, gráficas y comparación entre sesiones;
- documentación cronológica de decisiones técnicas.

8.3 PROBLEMAS ENCONTRADOS

Los problemas más relevantes fueron:

- curva de aprendizaje de Unreal Engine 5 y del sistema Chaos Vehicle;
- rigidez de soluciones basadas en NavMesh para vehículos físicos;
- complejidad de intersecciones, cruces y bordes de calzada;
- dificultad para diseñar un fitness que premie conducción útil sin favorecer atajos;
- coste computacional de entrenar poblaciones con renderizado y física activa;
- necesidad de depurar comportamientos emergentes no previstos, como estancamiento, marcha atrás o correcciones incorrectas.

8.4 LÍNEAS FUTURAS

Las líneas futuras recomendadas son:

- ampliar el entrenamiento específico en spawns problemáticos, especialmente aquellos con colisiones recurrentes;
- mejorar la percepción en intersecciones y la semántica de bordes;
- implementar perfiles de conductor normal, inseguro y agresivo;
- añadir interacción entre vehículos y reglas de prioridad más completas;
- extender el sistema a peatones, bicicletas y otros agentes autónomos;

- crear un modo de entrenamiento sin renderizado o desacoplado para acelerar generaciones;
- aumentar la duración y número de tests para estimar acierto con mayor confianza estadística;
- preparar un protocolo de validación con profesionales de ASPAYM antes de su uso con pacientes.

Agradecimientos

Pendiente de redacción definitiva por el autor. Se recomienda incluir a los tutores académicos, al equipo de HP SCDS, a ASPAYM y a las personas que hayan acompañado el desarrollo técnico y personal del proyecto.

Bibliografía

- [1] E. Romero Yuste, *CognitiveCarSimulator*, Trabajo de Fin de Grado, Grado en Desarrollo de Aplicaciones Interactivas 3D y Videojuegos, Escuela Politécnica Superior de Zamora, 2025.
- [2] L. Fuente, *CognitiveCarSimulatorReloaded - IA*, propuesta de proyecto, 2025.
- [3] Epic Games, *Unreal Engine 5 Documentation*. Documentación oficial de Unreal Engine.
- [4] Epic Games, *Chaos Vehicles*. Documentación oficial de Unreal Engine.
- [5] J. H. Holland, *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [6] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- [7] S. Haykin, *Neural Networks and Learning Machines*. Pearson, 2009.
- [8] Parlamento Europeo y Consejo de la Unión Europea, Reglamento (UE) 2016/679, Reglamento General de Protección de Datos, 2016.
- [9] UNE, *UNE 157001:2014. Criterios generales para la elaboración formal de los documentos que constituyen un proyecto técnico*.

Apéndice A

Fuentes analizadas

A.1 DOCUMENTACIÓN ADMINISTRATIVA Y DE FORMATO

Para la redacción de esta memoria se han revisado los documentos incluidos en INFO/UTILES. Entre ellos destacan:

- portada y normas básicas de estilo de la Escuela de Ingenierías Industrial, Informática y Aeroespacial;
- solicitud de preinscripción del TFG, con título, descripción y tutoría;
- resolución favorable de preinscripción de fecha 18 de mayo de 2026;
- propuesta *CognitiveCarSimulatorReloaded* - IA;
- memoria previa *CognitiveCarSimulator*;
- informe de seguimiento de prácticas HP SCDS;
- memoria final de prácticas del alumno;
- estructura orientativa propuesta por el tutor.

A.2 ACTAS DE REUNIÓN

Se han analizado las actas de seguimiento de 27/11/2025, 11/12/2025, 17/12/2025, 08/01/2026, 05/02/2026, 19/02/2026, 05/03/2026 y 19/03/2026. Estas actas permiten justificar las decisiones de arquitectura y los cambios de enfoque realizados durante el proyecto.

A.3 DESCRIPCIONES DEL PROGRAMA

Se han revisado descripciones fechadas de los módulos:

- EVOLUTIONMANAGER, con el gestor de generaciones, selección, mutación, persistencia y test;

- REDNEURONAL, con entradas, arquitectura de red, sensores, inicialización, mutación e inferencia;
- FITNESS, con cálculo de aptitud, penalizaciones, bonus y exportación de métricas;
- INTERSECTIONS, con semáforos, stops, cedas, zonas de cruce y validación normativa;
- OTROS, con macros auxiliares, raytracing, detección de estancamiento y backlog.

A.4 RESULTADOS EXPERIMENTALES

Los resultados documentados proceden de la carpeta `INFO/ANALISIS_DATOS/Análisis`. Para esta primera redacción se han tomado como referencia principal las sesiones:

- `train_2026-06-02_12-00`, con 607 generaciones y 19424 registros;
- `test_2026-06-02_13-17`, con 5 generaciones y 260 registros.

Las figuras incluidas en el capítulo de resultados proceden de los informes generados automáticamente por `analyse_current.py`.